

Programming Assignment 4 Report

Lizi Luo (lluo040) 862605965

Part 1: BTreeMap

File: `btree_map.h` `BTreeMap<K, V>` is a B-tree of order M (default 4, a 2-3-4 tree). Each node stores keys, values, and child pointers. Keys are sorted within each node and all leaves are at the same depth.

`HardDisk<T>` uses `BTreeMap<long, T>` to map sparse file indices to `PageFile` or `WordFile`. `SearchEngine` uses `BTreeMap<std::string, long>` to map URLs to page indices in sorted order. `std::unordered_map` is used for word-to-index mapping since it only needs $O(1)$ point queries with no ordering.

Insertion (Put)

Top-down splitting ensures no node is full when descending:

1. Empty tree: create a leaf root and insert.
2. Full root: create new root, make old root its child, split it, then insert.
3. Otherwise: call `InsertNonFull`.

`InsertNonFull` scans for the key position. If key exists, update value. If leaf, insert in place. If internal, split the child if full then recurse.

`SplitChild(parent, i)` splits `parent->children[i]` at median `mid = (M-1)/2`. Median key moves up to parent, right half becomes a new node.

Lookup and Traversal

`Search(node, key, idx)` scans keys left-to-right. If `keys[i] == key` return node and set `idx`. If leaf return `nullptr`. Otherwise recurse into `children[i]`.

`InOrderKeys` does in-order traversal: recurse into `children[i]`, output `keys[i]`, repeat, then recurse into the last child.

Part 2: PageFile, WordFile, SimpleBrowser

Files: `page_file.cc`, `word_file.cc`, `simple_browser.cc` `PageFile::Read/Write` and `WordFile::Read/Write` serialize structs line by line. The `<index>` line is handled by `HardDisk`.

`NormalizeUrl` strips the `http://` prefix, splits domain and path at first `/`, reverses domain parts using a stack, lowercases the result and removes trailing slashes.

`LoadPage` opens the file at the normalized URL path. Each line is parsed with `getline` on `" "` delimiter to extract `<a href="...">` URLs. The last non-empty line before `</body>` is split into keywords.

Part 3: Collect

File: `my_search_engine.cc` Collect does BFS over the web graph:

1. Enqueue all unvisited starting URLs.
2. Dequeue a page, load HTML via `SimpleBrowser`.
3. For each link: use `seen_urls` to skip per-page duplicates. Index new URLs and enqueue. Append link index to current page's `indices`.
4. For each word: use `seen_words` to skip duplicates. Index new words. Append current page index to word's `indices`.

`IndexPage` allocates a file index via `HardDisk::NewFile`, stores a `PageFile`, and records the URL mapping. `IndexWord` does the same for words.

Note: references from `BTreeMap::Get` are invalidated by subsequent `Put` calls (node splits). The code re-fetches references after any insertion.

Part 4: Rank

`Rank` initializes all pages with `impact = 1.0` and runs 20 iterations of `RankIteration`.

Each iteration: 1. Sum impact of dead-end pages, divide by total pages $\rightarrow$ `zero_link_impact`. 2. Each page distributes `impact / |links|` to each target's `impact_temp`. 3. Set `impact = impact_temp + zero_link_impact`, reset `impact_temp = 0`.

`impact_temp` is a write buffer so reads and writes don't interfere within one iteration.

Part 5: Search

`Search` finds pages containing all keywords and returns top N by PageRank.

1. Look up each keyword's `WordFile` to get its sorted page list.
2. Multi-way merge: if all pointers point to same index, record match and advance all. Otherwise advance pointers below the maximum.
3. Min-heap of size `num_results` keeps top matches by impact.
4. Extract results in descending order.

Complexity Summary

Operation	Complexity	Variables
<code>BTreeMap::Put / Get</code>	$O(\log n)$	n = number of entries
<code>BTreeMap::Keys</code>	$O(n)$	n = number of entries
Collect	$O(P \cdot (L+W) \cdot \log P)$	P = pages, L = links/page, W = words/page

Operation	Complexity	Variables
Rank (20 iterations)	$O(P + E)$	P = pages, E = total links
Search intersection	$O(K \cdot \text{sum}(P_k))$	K = keywords, P_k = pages per keyword
Search heap	$O(P_min \cdot \log N)$	P_min = min list length, N = results

Challenges

The main challenge was that `BTreeMap::Get` returns a reference that gets invalidated by a subsequent `Put` (node splits reallocate internal vectors). In `Collect`, after calling `IndexPage`, any held reference to a `PageFile` becomes dangling. Fix was to re-fetch the reference after every insertion instead of holding it accross calls.